

Introduction to python

* What is python?

→ Python is a interpreted object oriented general purpose high level programming language.

* Interpreted :- Python code is executed line by line by an interpreter at runtime rather than being compiled into machine code.

* Object oriented :- Python is built around the concept of objects & classes which allows you to organize code in a reasonable & modular way.

* General purpose :- Means it can be used for multiple applications such as data science, machine learning, desktop application, web application, scripts etc.

* High-level programming language :- means human understandable language i.e. human readable.

→ History of Python :-

* It was created by Guido van Rossum, and released in 1989.

* Guido van Rossum is fan of "MONTY PYTHON'S FLYING CIRCUS", this is a famous TV show in Netherlands.

* Named after monty python.

* Open Source can be downloaded from www.python.org.

- * First version of python released in 1991.
- * In 1994, python 1.0 was released with new Feature which map, filter, lambda.
- * Python 2.x also add some more Feature like comprehensions, garbage collection system.
- * After python 2.x, python 3.x released in 2008.
- * Latest version is python 3.11.6 (Released June 10, 2022)

Q) Why PYTHON?

- ⇒ *
- Python works on different platform (windows, mac, linux, raspberry pi, etc).
 - Python has a simple syntax similar to the English language.
 - Python has syntax that allows developers to write programs with fewer lines than some other programming language.
 - Python runs on interpreter system, meaning that code can be executed as soon as it is written.

* Applications of PYTHON :-

- NETWORK PROGRAMMING
- DATA ANALYSIS
- ROBOTICS
- WEBSITE & APPLICATION DEVELOPMENT
- DESKTOP APPLICATION
- GAMES DEVELOPMENT
- DATA VISUALIZATION
- SCIENTIFIC CALCULATION
- MACHINE LANGUAGE & ARTIFICIAL INTELLIGENCE
- 3D APPLICATION DEVELOPMENT
- AUDIO & VIDEO SOFTWARE DEVELOPMENT
- GOOGLE, NETFLIX, NASA, QUORA, FACEBOOK

Features of Python :-

Q) Simple :-

- * Python is a simple programming language.
- * When we read python program, we can feel like reading english statements.

- * It means more clarity and less stress on understanding the syntax of the language.

2] Easy to learn :-

- * Python uses very few keywords
- * Its programs use very simple structure.
- * Developing programs in python is easy

3] Open Source :-

- * There is no need to pay for python software.
- * Python can be freely downloaded from www.python.org ~~web~~ website.
- * Its source code can be ~~see~~ read, modified and can be used in programs as desired by the programmers.

4] High level language :-

- * High level languages use English words.
- * These are easy to learn & use like COBOL, PHP or java.
- * Python also uses English words in its programs.

5] Dynamically typed :-

* In python we need not declare data type for variables, whenever we are assigning the value based on value, type will be allocated automatically.

* But java, C etc are statically typed languages because we have to provide data type at the beginning only.

6] Platform independent :-

* When a python program is compiled using a python compiler it generates byte code.

* Python's byte code represents a fixed set of instructions that run on all operating systems and hardware.

* We can use python on almost all operating system like UNIX, LINUX, windows etc.

7] Portable :-

* Once a python program is written, it can run on any computer system using Python Virtual machine (PVM)

8] Procedure and object oriented :-

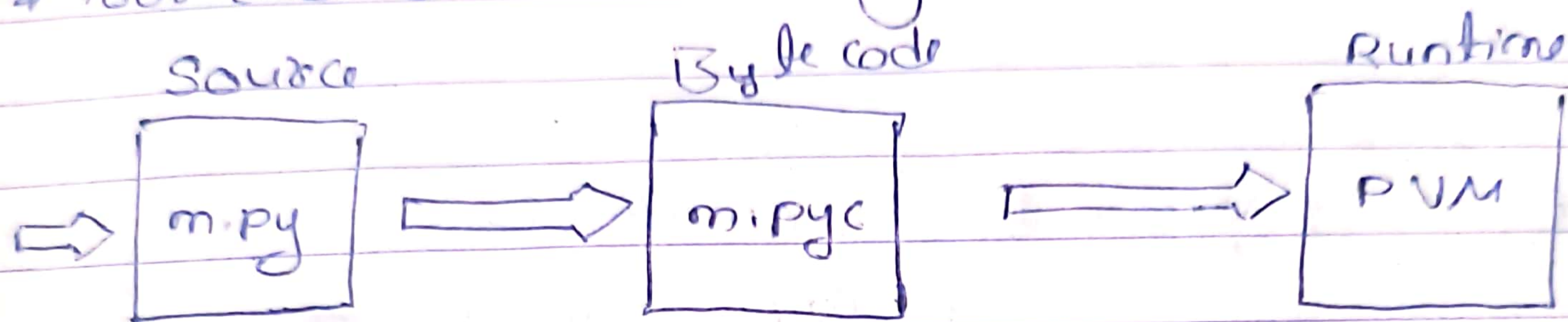
* Python is a procedure oriented as well as an object oriented programming language.

* In object procedure oriented programming language C ex:- C & pascal, the programs are built using fun & procedures.

* But in object oriented language C ex:- C++ & java, the programs use classes and objects.

WORKING WITH EXECUTION :- PYTHONPYTHON CODE EXECUTION:

- * Python's traditional runtime execution model :- Source code you type is translated to bytecode, which is then run by the python virtual machine (PVM).
- * Your code is automatically compiled, but then it is interpreted.



Source code extension is .py

Byte Code extension is .pyc (compiled python code).

There are two modes for using the python interpreter :-

- * INTERACTIVE MODE
- * SCRIPT MODE

Running Python in Interactive Mode :-

- * Without passing python script file to the interpreter, directly execute code to python prompt.
- * Once you're inside the python interpreter, then you can start
 >>> Print ("hello world")
 Hello world
- * Relevant output is displayed on subsequent lines without the >>> symbol.

U15K225S0003

① Write a python program to find area of a Circle.
where radius is ~~side~~ ^{radius} is r & π

$\Rightarrow r = 5$

$\pi = 3.142$

$area = \pi * r * r$

$print('area of the circle is :', area)$

~~area of the circle is = 78.55~~

Output:

Area of the circle is = 78.55

② Write a python program to calculate area of a Square.

$\Rightarrow a = 5$

$area = a * a$

$print('area of the square =', area)$

Output: ~~area of the square is = 25~~

③ Write a python program to check the given number is positive or not

$\Rightarrow$ if $num \geq 0$:

$print('Positive')$

else:

$print('Negative')$

$num = 5$

if $num >= 0$:

$print('positive')$

else:

$print('Negative')$

Output

pos/neg

4) Write a python program to find sum two given numbers.

$\Rightarrow a = 5$

$b = 10$

$sum = a + b$

$print('Sum of two no', sum)$

Output:

Sum of two no is 15

5) Write a python program to identify whether the given character is vowel or consonant.

$\Rightarrow ch = input('Enter a character:')$

if $ch.lower() in ['a', 'e', 'i', 'o', 'u']$:

$print('It is a vowel')$

else:

$print('It is a consonant')$

Output:

consonant

WSKZ25S0003

① Write a python program to check whether the given year is leap year or not.

⇒ years = 2024

if years % 4 == 0:

print("It is leap year")

else:

print("It is not leap year")

output:-

It is ~~not~~ leap year

② Write a python program to check the given number is even or odd

⇒ a = 6 n = input("Enter the number:")

~~if a % 2 == 0:~~ if n / 2 == 0:

print("The given number even")

else:

print("The given number odd")

③ Write a python program to check calculate simple interest.

⇒ P = 100

T = 12

R = 3500

SI = $P \times T \times R / 100$

print("SI = " + P + T + R + " ")

output:-

Simple interest = 60

④ Write a python program to display largest number.

⇒ num1 = 60

num2 = 30

if num1 > num2:

print("The given number is greater")

else:

print("The given number is greater")

Output:- num1

~~output:- num1 is greater~~

⑤ Write a python program to calculate the percentage by student by reading total mark

⇒ Marks = input("Total Marks")

marks = marks / 25 * 100

print("Total Marks:", marks)



Phosphorus

- ## Stringa

2 class 'style'

test :- Python

RUNNING PYTHON IN SCRIPT MODE:-

- the conduct of Mr. Ell.

- the name of the file

- to immediately

 $x = 2$

Output: 221

Print (2)

2

- + SINGLE-LINE COMMENTS Begin with a hash (#) symbol

SINGLE-LINE COMMENTS Begin with a hash (#) symbol and is useful in mentioning that the codeⁱⁿ should be considered as a command until the end of line.

- 67.

- $a = 10$ # assigning value to variable

the value of α and β is

$$C = a + b$$

Pairing (c)

Output: 1.30 ✓

DATATYPES IN PYTHON :-

- of memory and determining

- ## Data Types



Flooding compiler

Strong Turf 1st

1) Numbers :-

- * Number data type stores numerical values
- * This data type ~~stores numerical~~ is imm

- Pythagorean triples, indices, floating point numbers & complex numbers

July 2

- * They are all for
called just integers

1009

- ### Integras

61027

- with a decimal and
a fractional part

Complex

- Imagos of the term $a + bi$ where a & b are floats and i is imaginary
- It's the square root of -1 (which is an imaginary number)
- The real part of the number is a , and the imaginary part is b .

• They are positive or negative whole numbers with no decimal part.
Ex: -56

5698431L

Att. 5. 77

③ square root of -1 is complex number.

• Example ON int

```
a=10
print(a) #output: 10
print(type(a)) #output: <class 'int'>
```

• Example ON float

```
b=10.5
print(b) #output: 10.5
print(type(b)) #output: <class 'float'>
```

• Example ON complex

```
c=2+3j
print(c) #output: (2+3j)
print(type(c)) #output: <class 'complex'>
```

② BOOLEAN (True/False)

• In Python, boolean (bool) is a data type that represents one of two values.

• True

• False

Example:-

```
x=True
```

```
y=False
```

```
print(x)
```

```
#output: True
```

```
print(y)
```

```
#output: False
```

```
print(type(x))
```

```
#output: <class 'bool'>
```

③ Sequence :-

• A sequence is an ordered collection of items, indexed by position.

• It is a combination of mutable (value can be changed) & immutable (value cannot be changed) data types.

• There are three types of sequence data type available in Python, they are

i) Strings

ii) List

iii) Tuples

1) String

• A string in Python consists of a series or sequence of characters. String in Python are surrounded by either single quotation marks, or double quotation marks.

• 'Hello' is the same as "Hello".

• Individual character in a string is accessed by using a Subscript (index)

• Characters can be accessed using indexing and string operators.

• Strings are immutable, i.e. the contents of the string cannot be changed after it is created.

→ Example on string

```
print("Hello")
```

```
print('Hello')
```

```
output: =
```

```
Hello
```

```
hello
```

• OPERATIONS ON STRING

✓ Indexing → indexing the character

✓ Slicing → dividing

✓ Concatenation → joining multiple

✓ Repetition →

✓ Number ship →

Creating a string

```
>>> s="good morning"
```

• Indexing

```
>>> print(s[0])
```

```
>>> print(s[1])
```

```
>>> print(s[2])
```

```
>>> print(s[3])
```

```
>>> print(s[4])
```

```
>>> print(s[5])
```

```
>>> print(s[6])
```

```
>>> print(s[7])
```

```
>>> print(s[8])
```

```
>>> print(s[9])
```

Creating the list with elements or different data type.

• Accessing the item in the list

• Accessing the item in the position 2

• Displaying items from 2nd to last

• Displaying items from 1st position to 3rd

• Adding & removing the character

• Creating new strings concatenating multiple copies of the same string

in, not in
(membership
operator)

```
>>> s = "good morning"
>>> "m" in s
True
>>> "a" not in s
False
```

Using membership operator
check a particular character
in string or not. Return
True if present.

LISTS

* List is an Ordered sequence of items.

* Values in the list are called elements/items.

* It can be written as a list of comma-separated items.

(Values between square brackets [])

* Items in the lists can be of different data types

Operations on List :-

* Indexing

* Slicing

* Concatenation

* Repetition

* Update, insertion, deletion.

Ex :-

```
>>> my_list = [10, 20, 30]
```

```
>>> my_list[1] = 25
```

```
>>> print(my_list)
```

Creating a list

```
>>> list1 = ["python", 7.79, 101, "hello"]
```

```
>>> list2 = ["god", 6.78, 9]
```

```
>>> print(list1[0])
python
```

```
>>> list1[2]
101
```

```
>>> print(list1[1:3])
[7.79, 101]
```

Slicing (ending position-1)

creating the list with elements of different data type.

* Accessing the item in the position

* Accessing the item in the position

* Displaying items from start till end

Since operator is used to connect part of a string, or some part of a list

Python
Concatenation

* Repetition

Updating the list

Inserting element

Removing an element

```
>>> print(list1[1:3])
[7.79, 101, 'hello']
```

```
>>> print(list1 + list2)
['python', 7.79, 101, 'hello', 'god', 6.78, 9]
```

```
>>> list1 * 3
['god', 6.78, 9, 'god', 6.78, 9, 'god', 6.78, 9]
```

```
>>> list1[2] = 45
>>> print(list1)
['python', 7.79, 45, 'hello']
```

```
>>> list1.insert(2, "python")
>>> print(list1)
['python', 7.79, 'python', 45, 'hello']
```

```
>>> list1.remove(45)
>>> print(list1)
['python', 7.79, 'python', 'hello']
```

* Adding & printing the items of the list

* Displaying items from the position till the last

* Adding & printing the items of the list

Creates new strings, concatenation

multiple copies of the same string

Updating the list using index value

Inserting element at end position

Removing element by giving the element directly

3) Tuple :-

* Tuple is same as list, except that the set of elements is enclosed in parentheses instead of square brackets.

* A tuple is an immutable list. i.e. once tuple has been created you cannot add elements to a tuple or remove elements from the tuple.

Benefit of tuple :-

* Tuples are faster than lists

* If the user wants to protect the data from accidental changes, tuple can be used.

If you try to access a key which doesn't exist, you will get an error message.

```
>>> words = {'house': 'house', 'cat': 'kitten'}
>>> words['cow']
```

TypeError (most recent call last):

```
File "List 28500003.py", line 1, in <module>
```

```
words = {'cow': 'cow'}
```

creating a dictionary - Assignment 6

```
student1
```

```
name = "Sourya",
```

```
age = 25,
```

```
course = "IT"
```

```
}
```

Accessing values

```
print(student1['name']) # Output: 'Sourya'
```

```
print(student1['age']) # Output: 25
```

Adding a new key-value pair

```
student1['grade'] = 'A'
```

```
Print (student1)
```

```
# Output: {'name': 'Sourya', 'age': 25, 'course': 'IT', 'grade': 'A'}
```

```
grade: 'A'}
```

VARIABLES, EXPRESSIONS AND STATEMENTS:

Values & Types:

* A value is one of the basic things a program works with.

* It may be like 2, 10.5, "hello" or,

each value in python has a type.

* Type of 2 is integer; type of 10.5 is floating point number; "hello" is string etc.

* The type of value can be checked using type function as shown below

```
>>> type('hello')
```

```
<class 'str'>
```

```
>>> type(3)
```

```
<class 'int'>
```

```
>>> type(10.5)
```

```
<class 'float'>
```

```
>>> type(15.9)
```

```
<class 'float'>
```

In the above four examples, one can notice that various types str, int & float.

* Observe the 4th example - it clearly indicates the whatever enclosed with

- in a double quote is a string.

ASSIGNMENT - 1

1) Slicing in list:

```
>>> list1 = ['Python', 7.79, 101, 'hello']
```

```
>>> list2 = ['good', 6.789]
```

```
Print (list1[2])
```

```
# Output = ['Python', 7.75, 101]
```

```
Print (list1[2])
```

```
# Output = ['101', 'hello']
```

2) Removing an element is list

```
>>> list1.remove(101)
```

```
Print (list1)
```

```
['Python', 7.79, 'hello']
```

Using index value

list.remove()

Index value will not support.

3) Inserting an element in list
 list = ['Rishi', 'Bhagy']

list.insert(0, 'Hille')

Output: ['Hille', 'Rishi', 'Bhagy']

list.remove('Bhagy')

Output: ['Hille', 'Rishi']

Output: error

4) Set is a union

a = {1, 2, 3, 4, 5}

b = {3, 4, 5}

print(b.union(a))

Output: {1, 2, 3, 4, 5}

5) Dictionary adding new key
 Student = {'name': 'Bhagy'}

age = 25

Student['age'] = 25

print(Student['age'])

Output: 25

print(Student['grade'])

Output: error

VARIABLES

* A variable which helps to store a value in the program
 * Variable may take value that can be modified where required in the program.

* Note that, in Python, a variable need not be declared with a specific type before its use.

* Whenever we use a variable, just use it. The type of it will be decided by the value assigned to it.

* A value can be assigned to a variable using assignment operator (=).

* Assigning value to variable:

* Value should be given on the right side of assignment operator (=) and variable on left side.

Ex: >>> counter = 45

print(counter)

* Assigning multiple values to several variables simultaneously:

Ex: >>> a = b = c = 100

* Assigning multiple values to multiple variables:

Ex: >>> a, b, c = 25, 4, 'exam' // Upcoding

IMP Rules for Python Variables:

* A variable name must start with a letter or the underscore character.

* A variable name cannot start with a number.

* A variable name can only contain alpha-numeric characters & under

-scores (a-z, 0-9)

* Which is usually used to combine variables with join words like

my_salary, student_name etc. no other special characters like @, # etc are allowed

Operator: An Operator

- Special symbols used to indicate what to do on
- A. operator can work on single operand (unary operator)
- B. two operands (binary operator)
- There are several types of operators. They are:
 - Arithmetic operators, logical operators, comparison operators, assignment operators, and bitwise operators.

Operator	Meaning	Example
+	Addition	Sum = a + b
-	Subtraction	Diff = a - b
*	Multiplication	Pro = a * b
/	Division	Q = a / b
//	Floor Division - returns only integer part after division	$x = 5 / 3$ 1.666666667
%	Modulus remainder after division	$x = 5 \% 3$ 2
**	Exponent	$c = x ** y$ (Power of y)

- Relational (or) Comparison operators are used to check the relationship (less than, greater than etc) between two operands.
- These operators return a boolean value - either true or false.

- Assignment operators :- apart from simple assignment operator (=) which is used for assigning values to variable, python provides compound assignment operators.

For example :-

$$x = x + y \text{ can be written as } x += y$$

- Now, += is compound assignment operator.
- Similarly one can use most of the arithmetic and bitwise operators (only binary operators, but not unary) like +, -, *, /, %, //, & etc. As compound assignment operators.

Example :-

$$x = 5$$

Expressions :-

- A combination of values, variables and operators is known as expression.

- Following are few examples of expression -

$$x = 5$$

$$y = x + 10$$

$$z = x - y * 3$$

- The python interpreter evaluates simple expressions & gives results even without print ().

Example :-

$$>>> 5$$

$$5 \quad \# \text{ displayed as it is}$$

$$>>> 1 + 2$$

$$3 \quad \# \text{ displayed the sum}$$

$$15 + 20 = 35 + 6$$

$$(3 + 4) \times 5 = 35 \rightarrow 41$$

$$\frac{15}{35} \times \frac{15}{35} = \frac{225}{1225}$$

ORDER OF OPERATIONS :-

- When an expression contains more than one operator, the evaluation of operators depends on the precedence of operators.
- The python operators follow the precedence (which can be remembered as per below).

→ Powers have the highest precedence in any expression.

✓ The operations within parentheses will be evaluated first.

✓ For example, in the expression $(a + b) \times c$, the addition has to be done first and then the sum is multiplied with c .

→ Exponentiation has the end precedence.

✓ But, it is right associative. That is, if there are two exponentiation operations continuously, it will be evaluated from right to left.

(unlike)

For example:

```
>>> print (2**3) * 4
16
>>> print (2**3) * 4
16
>>> print (2**3) * 4
16
```

• Multiplication & Division are the next priority out of the arithmetic operations, whichever comes first in the expression is evaluated.

```
>>> print (5 * 4 / 2)
10
>>> print (5 / 2 * 4)
10
```

• Addition & Subtraction are the least priority out of the two operations, whichever appears first in the expression is evaluated. i.e., they are evaluated from left to right.

STRING OPERATIONS :-

✓ String concatenation can be done using + operator as shown below.

```
>>> x = "32"
>>> y = "us"
>>> print (x + y)
32us
```

observe the output :- here, the value of y (a string "us", but not a number us) is placed just in front of value of x (a string "32"). Hence the result would be "32us" & its type would be string.

• Note :- One cannot single quote to enclose a string value, instead of double quote.

ACCEPTING USER INPUT :-

• Python uses the built-in function `input()` to read the data from the keyboard.

• When this function is invoked, the user input is captured. The output is read till the user presses enter-key.

For example:

```
>>> user = input()
hello how are you?
>>> print (user)
hello how are you?
```

• When input is function is used, the cursor will be moving to receive the data.

• For a better understanding, it is better to make a prompt message for the user informing what needs to be entered as input.

CONDITIONAL EXECUTION

- In general, the statements in a program are to be executed sequentially.
- But, sometimes we need a set of statements to be executed based on some condition.

Boolean expressions

- A Boolean expression is an expression which results in ^{or false} true or false.
- The true or false are special values that belong to the class ^{or false} bool.

What is following

>>> type(1 < 2)
 < bool: bool
 >>> 1 < 2
 True
 < bool: bool
 >>> 1 < 2
 True

OPERATOR

MEANING

EXAMPLE

>	Greater than	$a > b$
<	Less than	$a < b$
>=	Greater than or equal to	$a >= b$
<=	Less than or equal to	$a <= b$
=	Comparison	$a == b$
!=	Not equal to	$a != b$
is	is same as	$a is b$
is not	is not same as	$a is not b$

Logical Operators

There are 3 logical operators in Python as shown in Table

Operator

and

Meaning

Returns true, if both operands are true

Example

$a \& b$

or

Returns true, if any one of two operands is true

$a \& b$

not

Returns true, if the operand is false (it is a unary operator)

$a \& b$

NOTE:

- 1) Logical operators treat the operands as boolean (true or false).
- 2) Python treats any non-zero number as true & zero as false.
- 3) While using & operator, if the first operand is false, then the second operand is not evaluated & by python. Because false and 'ed with anything is false.
- 4) In case of or operator, if the first operand is true, the second operand is not evaluated. Because true and with anything is true.

INDENTATION

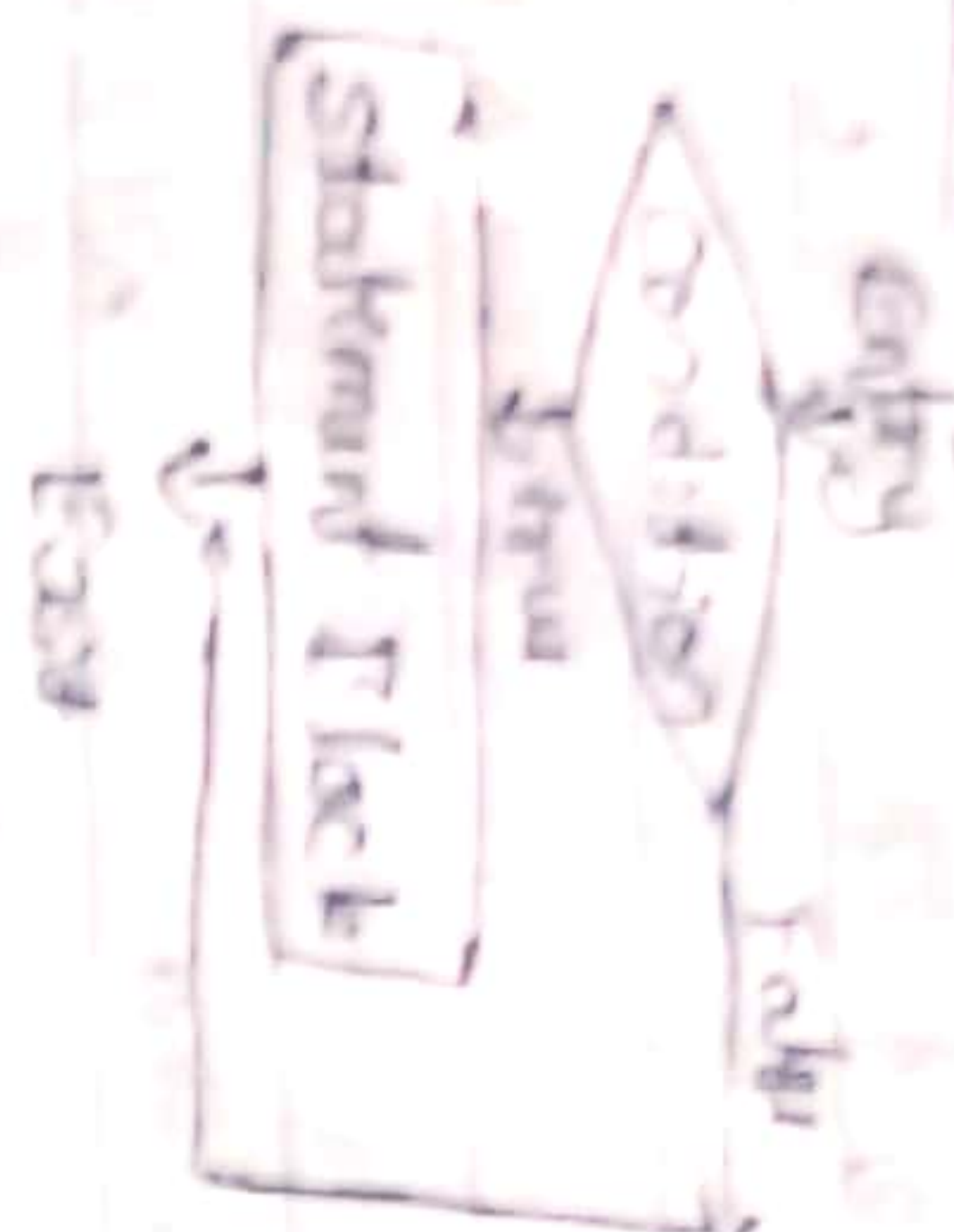
- Most of the programming languages like C, C++, Java use braces {} to define a block of code. But, python uses indentation.
- Blocks of code are denoted by line indentation.
- It is a space given to the block of code for code for class and function definitions or flow control.

Conditional execution:-

- The basic level of conditional execution can be achieved in Python by using if statement.
- The syntax and its flowcharts are as below.

If condition:

Statement block



* Choose the colon symbol after condition.

* When the condition which is a boolean expression is true, the statement block will be executed.

* Otherwise, it is skipped.

* A set (block) of statements to be executed under it as:

decided by the indentation (tab space) given

Ex: ~~age > 18:~~ age = 35

if age > 18:

 print ("eligible to vote")

* Usually, if conditions have a statement block.

* In any case, the programmer fails to do nothing when the condition is true, the statement block can be skipped by just typing pass statement as shown below.

Ex: if x < 0:

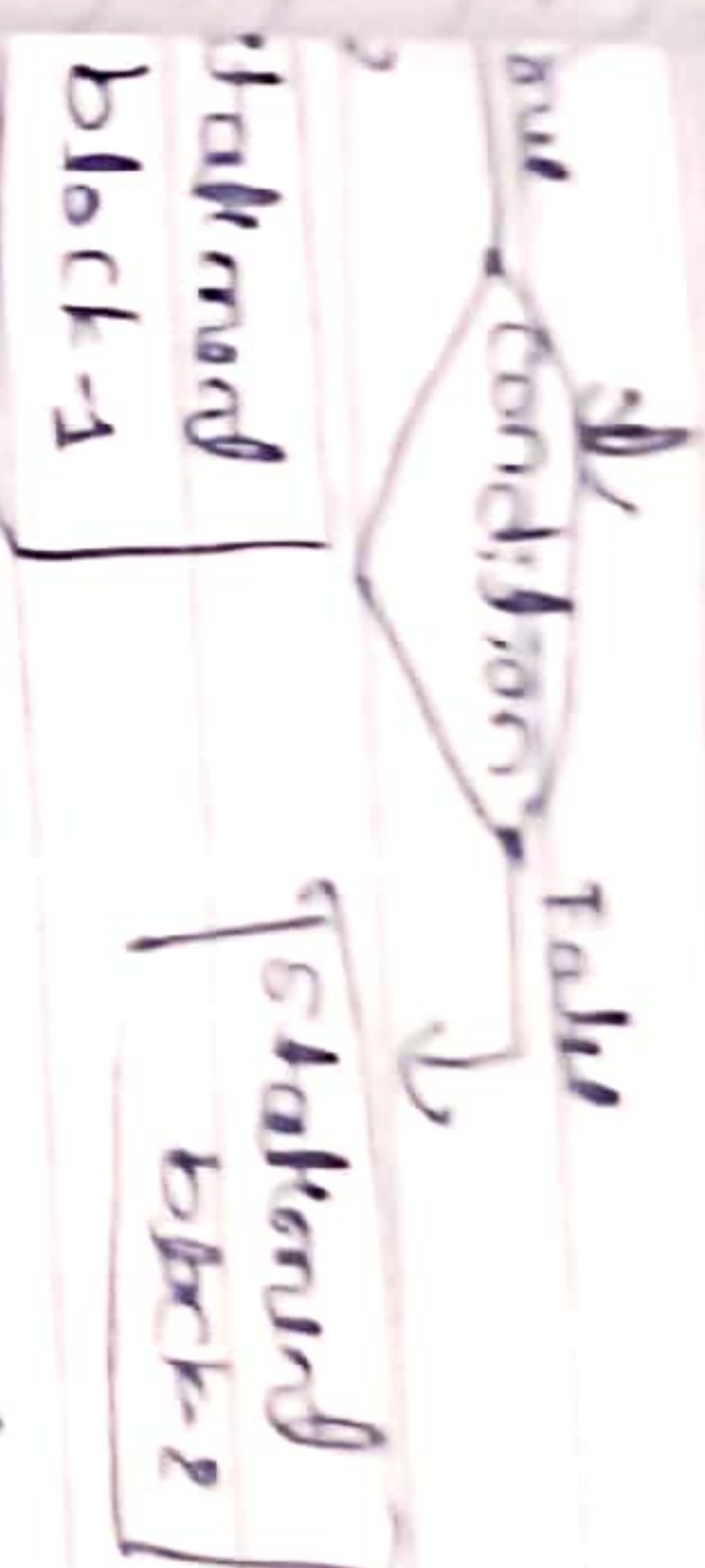
 pass # do nothing when x is negative.

Alternative Execution:

* A second form of if statement is alternative execution, in which there are two possibilities based on condition evaluation.

* Here, when the condition is true, one set of statements will be executed and when the condition is false, another set of statements will be executed.

* The Syntax & flowchart are as given below.



* These two alternatives are known as branches.

Ex: age = 2

if age > 18:

 print ("eligible for vote")

else:

 print ("Not eligible for vote")

Nested and conditional

* The conditional statements can be nested

* That is, one set of condition as statements can be nested inside the other.

* It can be done in multiple ways depending on program requirements.

Ex: if user == "admin":

 print ("Access granted")

else:

 print ("Wrong password")

else:

 print ("Unknown user")

Chained Conditions:

* Some of the programs require more than one possibility to check for deciding set of statements.

* That means, we may have more than one branch. This is solved with the help of chained conditions.

Syntax:

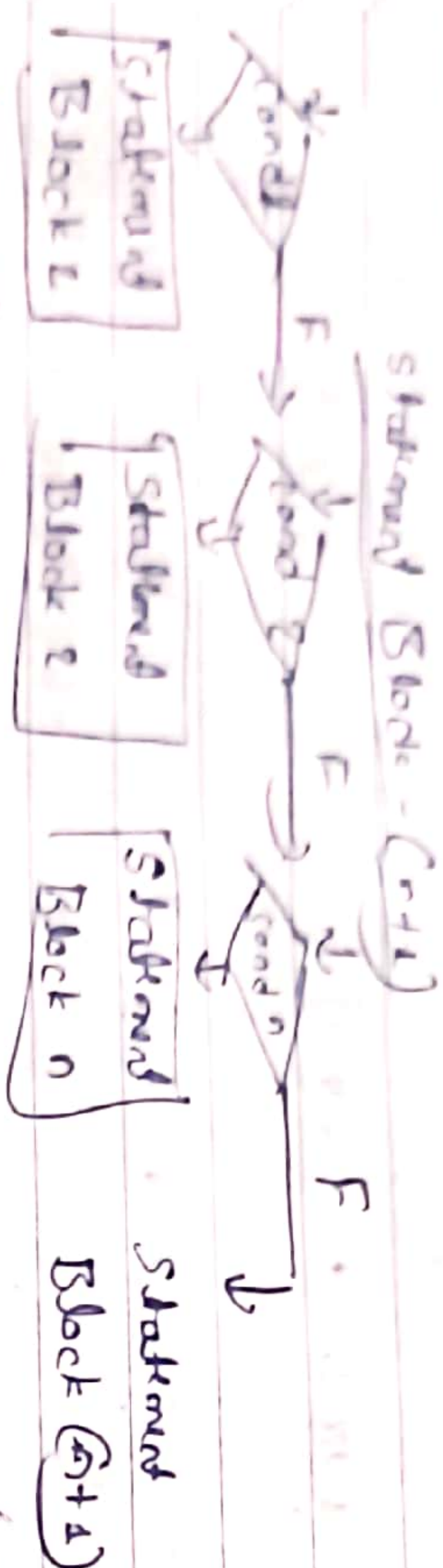
if condition 1:

 Statement Block-1

if Condition 1:
Statement Block-1

else Condition-2:
Statement Block-2

else



for if marks >= 90:

print ("a grade")

elif marks >= 75:

print ("b grade")

else:

print ("c grade")

Iterations

- Computers are often used to automate repetitive tasks, repeating identical or similar tasks without making errors.

- Something that computers do well and people do poorly.
- Repeated execution of a set of statements is called "iteration".

Updating Variables

- A common pattern in assignment statements is on assignment statement that updates a variable, where the new value of the variable depends on the old.

$x = x + 1$

"This means get the current value of x , add 1, and then update x with the new value."

- If you try to update a variable that doesn't exist, you get an error, because Python evaluates the right side before it assigns a value to x .

$x = x + 1$

Name Error: name ' x ' is not defined

- Before you can update a variable, you have to initialize it, usually with a simple assignment.

$x = 0$

$x = x + 1$

Updating a variable by adding is called an increment, subtraction is called Decrement.

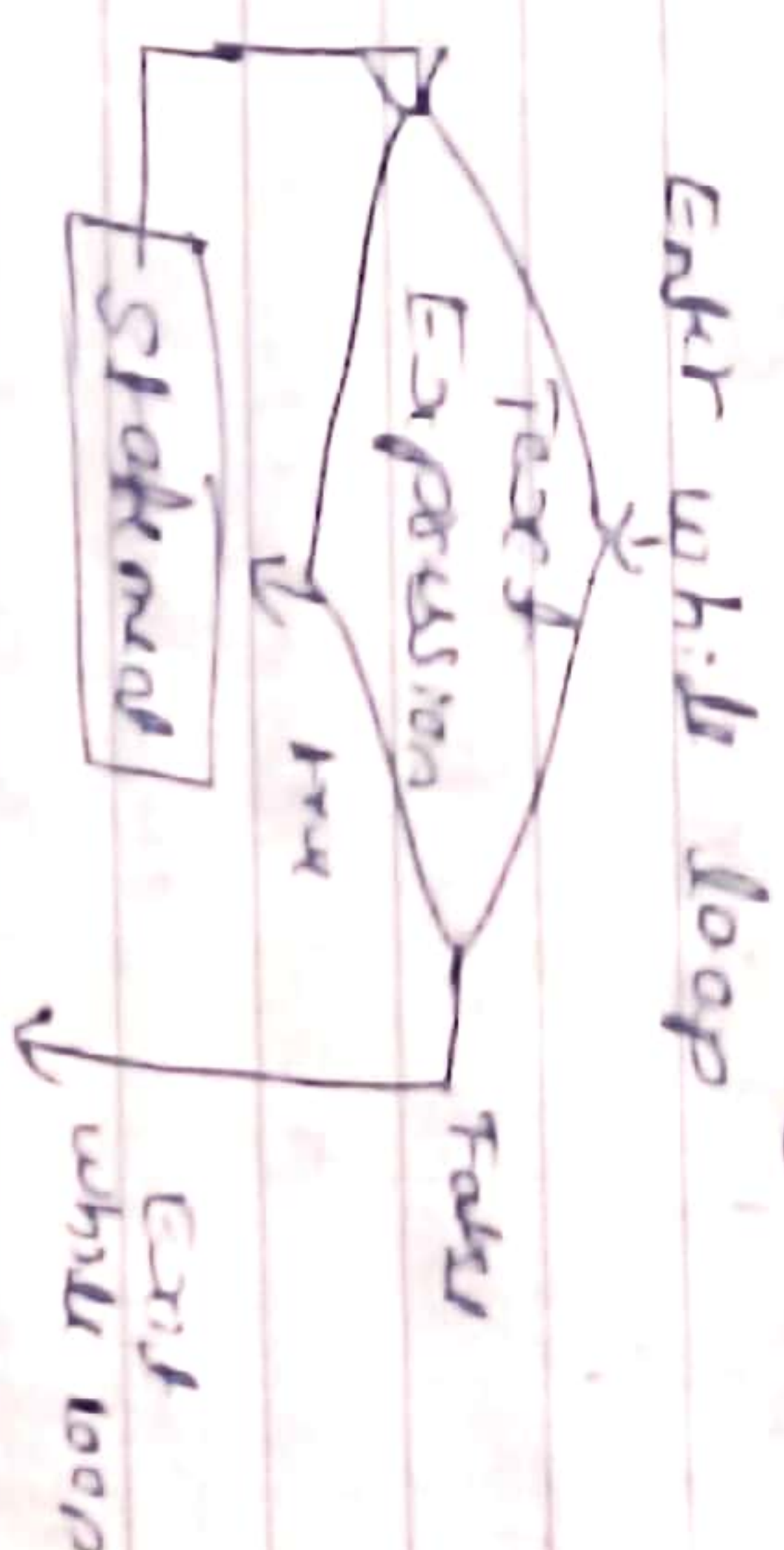
The while loop

While loop is used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed.

Syntax:-

while expression:

statement(s)



Python example for

while loop

Count = 0

while (Count < 3):

Count = Count + 1

print ("Hello BCR")

output: Hello BCR

Hello BCR

Hello BCR

Do-while loops

- The value of the condition is always true. Therefore, the body of the loop is run infinite times until the memory is full.
- It is an infinite loop because there is no iteration variable telling you how many times to execute the loop.

Ex: `do {`
`// body`
`while (condition);`

while condition is always true

print (do-while loop)

Output: Infinite loop

1st loop

2nd loop

ending

Finishing statement with continue :-

The code statement skips the current iteration & moves to the next one, without executing the remaining code inside the loop for that iteration.

SYNTAX

while condition;

Code before continue

if some condition;

continue

Example:

Code after continue (this will be skipped if condition runs)

i = 0

while i < 5;

 i++

 if i == 3;

 continue

 print(i)

Output: 1
2
3
4
5

DEFINITE LOOPS USING FOR

- Sometimes we want to loop through a set of things such as a list of words, the lines in a file, or a list of numbers.
- When we have a list of things to loop through, we can construct a definite loop using a for statement.
- We call the while statement an infinite loop because it simply loops until some condition becomes false, whereas the loop is looping through a known set of things so it runs through as many iterations as there are things in the set.

Syntax:-

for variable in sequence:

body of loop

* Variable: a range, list, string, or other iterable

* Sequence: a range, list, string, or other iterable

Example 1:- PRINT NUMBERS FROM 1 TO 5

for i in range(1, 5):

 print(i)

Output: 1
2
3
4

Example 2:- ITERATE THROUGH CHARACTERS IN A STRING

for ch in "Python":

 print(ch)

Output: P
y
t
h
o
n

Example 3:- LOOP THROUGH A LIST

fruits = ["apple", "banana", "cherry"]

for fruit in fruits:

 print(fruit)

Output: apple
banana
cherry

Cherry

FUNCTIONS:

- * Functions are building blocks of any programming language.
- * A group of instructions intended to perform a specific independent task is known as 'fun'.
- * Fun is a sub program which consists of set of instructions used to perform a specific task. A large program is divided into basic building blocks called 'fun'.

Need for function:

- * When the program is too complex & long they are divided into parts. Each part is separately coded & combined into single program. Each subprogram is called as 'fun'.
- * Debugging testing & maintenance becomes easy when the program is divided into subprogram.
- * Fun are used to avoid ~~repetitive~~ re-writing same code again & again in a program.
- * Fun provides a code re-usability.
- * The length of the program is reduced.
- * Syntax of the fun Declaration & Definition.
 - * A fun is defined using the def keyword, followed by a fun name & parentheses.

Syntax: def my_function():

print('hello from a function')

This creates a fun named my_fun that prints 'hello' when called.

Function calls

- * A fun is a named sequence of instructions for performing a task.
- * When we define a fun we will give a valid name to it & fun specifies the instructions for performing required task.
- * Later, whenever we want to do that task, a function is called by its name.
- Ex: >>> type (is)
<<< is int
- * Here type is a fun name, is is the argument to a fun and <<< is int is the result of the fun.
- * Usually, a fun takes zero or more arguments & returns the result.

Build in fun:-

- * Python provides a rich set of built-in fun for doing various tasks.
- * The programmer/developer need not know the internal working of these fun; rather, they need to know only the purpose of such fun.
- * Some of built in fun are given below -
- * max(): this fun is used to find maximum value among the arguments. It can be used for numeric values or even to strings.
- Ex: MAX (10, 20, 30, 40, 50) # maximum of 5 integers is 50
- * Min(): as the name suggests, it is used to find minimum of arguments.
- Ex: min (10, 12, 14, 16, 18) # minimum of 5 integers is 10
- * len(): this fun takes a single argument & finds its length. The argument can be a string, list, tuple etc.

to: King Chillo how are you?")

Type conversion functions:-

- As we have seen earlier, while discussing input in `Python`, the type of the variable/value can be converted using `int()`, `float()`, `str()`.
- `Python` provides built-in functions that convert values from one type to another. Consider following two examples.

The `int` function takes any value & converts it to an integer, if it can, or complains otherwise.

- `int('100')` $\rightarrow$ integer enclosed within single quote $\rightarrow$ converted to integer type.
- `int('100.1')` $\rightarrow$ integer enclosed within double quote $\rightarrow$ actual string cannot be converted to `int`.
- Trace back (most recent call first):
- File "`<ipython>`", line 1, in `<module>` `int('100.1')`
- ValueError: invalid literal for int() with base 10

Math Fun:

`Python` provides a rich set of mathematical functions through the module `math`.

- To use these functions, the `math` module has to be imported in the code. Some of the important functions available in `math` are given here under.

For

`sqrt()` - This function takes one numeric argument & finds the square root of that argument.

```
>>> math.sqrt(36)
Out[1]: 6.0
>>> math.sqrt(2.5)
Out[1]: 1.58113883008019
```

`pi` - The constant value `pi` can be used directly whenever we require.

```
>>> print(math.pi)
3.141592653589793
```

`log` - This function is used to find the logarithm base 10, of a given number.

```
>>> math.log(10)
0.3010299956639812
```

`log10` - This is used to compute logarithm (base 10) of a given number.

```
>>> math.log10(10)
0.6931471805599453
```

`sin` - as the name suggests, it is used to find sine value of a given argument. Note that, the argument must be in radians (not degrees). One can convert `90` or `degrees` into radians by multiplying `pi/180` as shown below.

```
>>> math.sin(90 * math.pi/180)
Out[1]: 1.0
```

`cos` - used to find cosine value. `>>> math.cos(0.7071067811865476)`

`tan` - function to find tangent of an angle, given as argument. `>>> math.tan(0.7071067811865476)`

`pow` - This function takes two arguments `x` & `y`, then finds the power of `y`. `>>> math.pow(5, 2)`

Python has a module random for the generation of random numbers. One has to import this module in the program.

The function rand() is also random().

Example:-

import random

or

import random

num = random.random()

print random.random() * 10

print(num)

RECURSION: The call itself only.

Imagine you want to know the name of a person in the house that you are standing in. You ask people to find out about it.

They keep asking the next person until they find an answer. Once an answer is found, they send it back until it reaches you.

The above examples depict a process in which a particular action is repeated until a condition is met.

A process in which a function calls itself is called recursion.

This process helps ease the method of solving problems by replacing iterative code with recursive statements.

Recursion is used in Python when problems can be broken into simpler parts for easier computation & more readable code.

Types:-

1) BASIC CASE: This helps us to determine the recursive function. It is a simple case that can be answered directly without recursion. If satisfied, it returns the final computable answer. If this is omitted, the function will run until it runs.

Python interpreter limits the number of recursive calls for a function to 1000 by giving a recursion error.

2) GENERAL (RECURSIVE) CASE:- This case uses recursion.

If it is called unless the base condition is satisfied.

Syntax:-

def rec_func_name():
if (condition):
base case

Simple statement without recursion

else: # general case

statement calling rec_func_name()

Ex:-

def factorial(n):

if n == 0 or n == 1: # base case

return 1

else:

return n * factorial(n-1) # recursive call

Output:-

100

num = 5
result = factorial(num)

print("Factorial of num is", result)

SCOPE: Local and global variables:-

The scope defines the accessibility of the Python Object. To access the particular variable in the code,

The scope must be defined as it cannot be accessed from any where in the program.

A variable is only available from inside the region it is created. This is called scope.

Based on the scope, we can classify Python variables into two types.

1) Local variables

2) Global variables

3) Local variable:

- When we declare variable inside a function, the variable will have a local scope (within the function) but cannot access this outside the function.
- Another type of variable is called local variable.
- ex: `def greet():`
`print('hello')`
`print(greet())`

Output:

local hello

nameerror: name 'message' is not defined.

`global` is a key to access message variable

is outside `greet()` function

`print(message)`

3) Global Variable:

- In python, a variable declared outside of the function or in global scope is known as a global variable.
- This means that a global variable can be accessed inside or outside of the function.

`message = 'hello'`

`def greet():`

`print('local', message)`

`print('global', message)`

Output:

local hello

global hello

MODULES:

- A module is a file containing python definitions, functions, statements & instructions.
- Standard library of python is extended as modules.
- To use these modules in a program, programmer needs to import the module.
- Once we import a module, we can reference or use the objects, functions or variables inside it.

- There is large number of standard modules also available in python.
- Standard modules can be imported in any way as we import our user defined modules.

- Every module contains many functions.
- To access or use function you have to specify the name of the module & the name of the function separated by `dot`. This format is called `dot notation`.

Syntax:

`import module_name`

`module_name.function_name (variable)`

2) Name Error

Importing Built-in Module:

`import math`

`x = math.sqrt(25)`

`print(x)`

Importing user Defined Module:

`import cal`

`x = cal.add(5, 10)`

`print(x)`